



Pokerbots 2026

Lecture 11: How to Write Performant Code (Part 2)

Sponsors



CITADEL | CITADEL Securities





Announcements

Make-up Progress Reports due TODAY

- As per syllabus, if your team missed the Week 3 bot submission deadline, a progress report is required to earn credit
- Reports are half a page and must list team name and members
- Submit via Canvas by 11:59pm Tuesday (Today)

HACKATHON PART 2

- We had food
- We had prizes
- We had snacks/drinks

You just had to be there

Lightning Tournament #2

1. hey0999	\$400
2. double crossers	\$300
3. Policy Gradient Is All You Need	\$225
4. horseshoes	\$175
5. Los Camellos	\$150

Week 3 Mini Tournament Results

WINNER [\$1000]:

double crossers

BIGGEST UPSET [\$500]:

don't panic

Tournament ELO 818, won majority matches against "Three of a Kind" (ELO 1717), 30-0!

MOST IMPROVED [\$750]:

Beta Punters

Tournament ELO 1386 → 1915



This Week...

Monday 1/26

(cancelled due to snow storm)

Hopefully no one showed up to 6-120

Tuesday 1/27

Lecture 11: How to Write Performant Code (Part 2)!

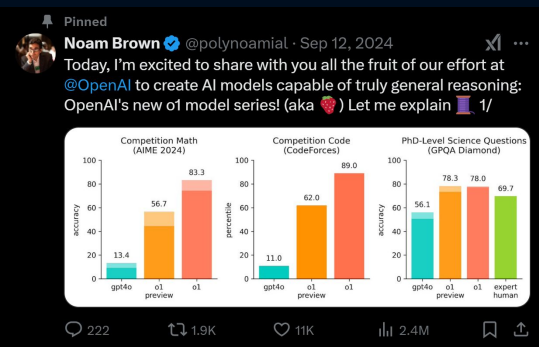
Team Strategy Reports due 11:59PM EST

- 3-5 pages, double spaced, ≥ 1 member must submit
- Assignment details and submission on Canvas: pkr.bot/canvas
- Description also available in syllabus: pkr.bot/syllabus

Wednesday 1/28

Guest Lecture: Dr. Noam Brown

- Won Pokerbots and co-created Deep CFR while PhD at CMU
- Created Libratus and Pluribus, the world's first superhuman poker AIs
- Research scientist currently at OpenAI, previously worked at Meta AI
- Leading role in developing OpenAI's latest GPT o1 and o3 LLMs



Wednesday 1/28 (cont.)

Final Bot Submission due 11:59PM EST

- Upload and select bot as active on scrimmage server
- Both report and bot needed to pass this class!
- Bot will compete in last and final Pokerbots tournament
- Non-secret prize amounts listed on syllabus

Final Tournament Prizes	
First place	\$10,000
Second place	\$6,500
Third place	\$3,500
Fourth place	\$2,000
Fifth place	\$1,000
First place in language (Python, Java, or C++)	\$500 x 3
Second place in language (Python, Java, or C++)	\$250 x 3
Third place in language (Python, Java, or C++)	\$125 x 3
Best freshman-majority (>51%) team	\$2,000

Friday 1/30

Pokerbots Final Event 4:30-7PM in Tang Center (E51)

- Presentation of Awards
- Closing Ceremony
- Sponsor Event and Puzzle Hunt
- Lots of free merch and raffle prizes!
- Dinner provided 😊

DATE TBD (probably ~beginning of February)

Epic Poker Tournament!

- Location TBA
- \$3500 cash prize pool

A large, stylized flower graphic composed of several overlapping circles in various shades of blue, located on the left side of the image. The text "Giveaway Game!" is centered horizontally and partially overlaps the right side of the flower graphic.

Giveaway Game!

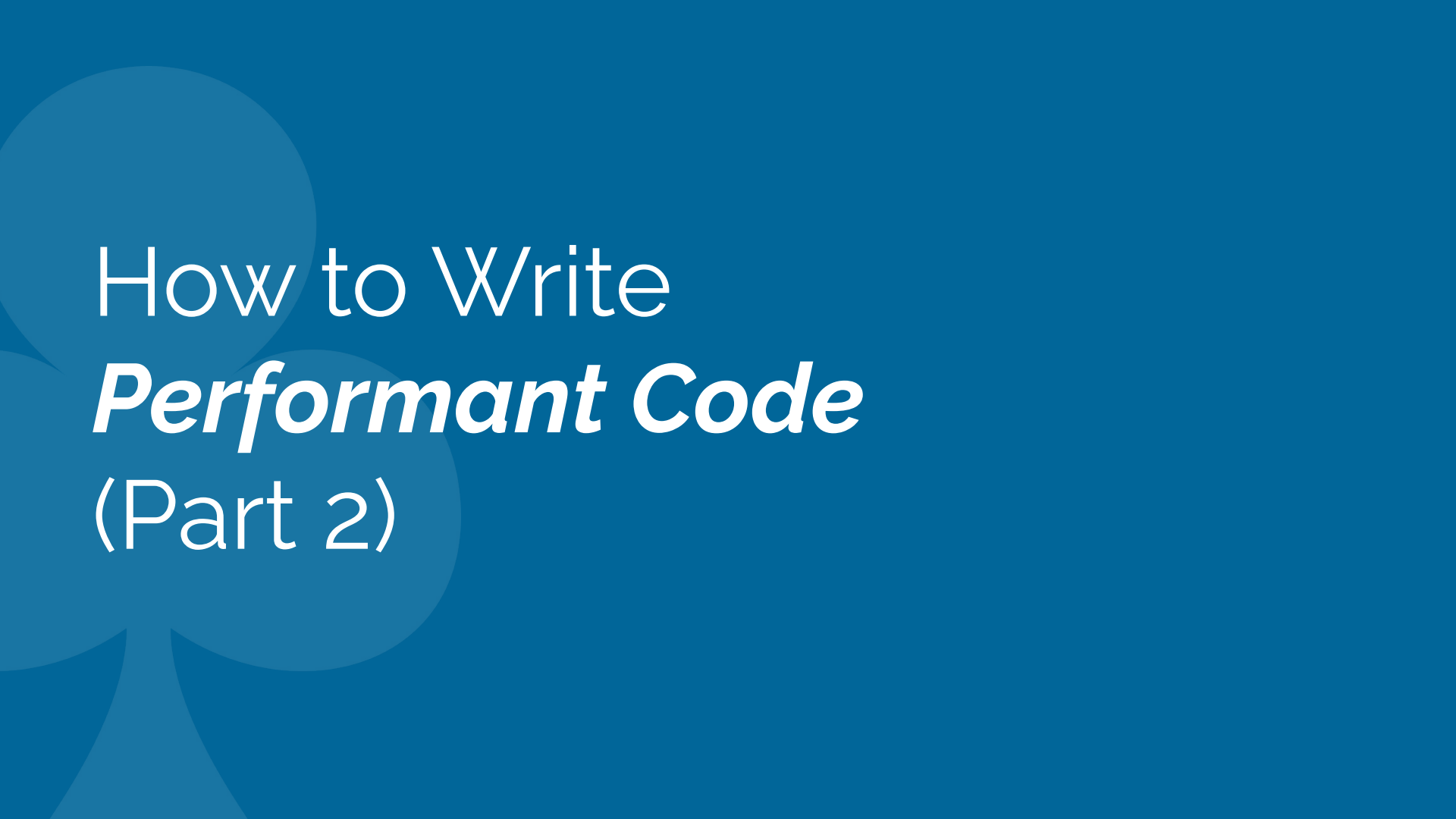
I Lost N...

- I have a piece of Python code I ran, but I lost the value of N 😞
 - For my lost N, the code ran in ~59.88 seconds
 - I remember that for $N = 10000$ my code ran in 0.0400 seconds!
- Goal: Guess my N (winner is whoever's value of N results in runtime closest in absolute difference 59.88 seconds)
 - (Ties broken by earliest submission)
- Prize: LG 24" Monitor

Questions?

pkr.bot/ilostn

```
def f(n):  
    s = 1  
    for i in range(n):  
        if i % 2 == 0:  
            s *= i * 3 + 1  
        else:  
            s *= i * 5 + 2  
    return s  
N = ???  
x = f(N)
```


The background is a solid blue color with several overlapping, semi-transparent circles of varying shades of blue, creating a layered effect. The text is white and positioned on the left side of the image.

How to Write *Performant Code* (Part 2)

Agenda

- How Do We Improve Speed?
- Addressing the True Problem
- Locality
- Dealing with a Bottleneck
- Takeaways
- Practical Example



How Do We Improve Speed?

Recall Assumptions

1. We run our code on some “processor”
2. Our processor can execute “instructions” from a given set (i.e. addition, memory access, etc)
3. Our code can easily be transformed into a set of these “instructions”
4. The time it takes to execute each different “instruction” is roughly similar

Addressing Assumptions

1. We run our code on some “processor”
2. Our processor can execute “instructions” from a given set (i.e. addition, memory access, etc)
3. Our code can easily be transformed into a set of these “instructions”
4. The time it takes to execute each different “instruction” is roughly similar

Assembly

Some instructions:

What are these?

A simpler 'programming language'

Instructions designed to be performed directly on our hardware!

Format	Instruction	Opcode	Funct3	Funct6/7
R-type	add	0110011	000	0000000
	sub	0110011	000	0100000
	sll	0110011	001	0000000
	xor	0110011	100	0000000
	srl	0110011	101	0000000
	sra	0110011	101	0000000
	or	0110011	110	0000000
	and	0110011	111	0000000
	lr.d	0110011	011	0001000
	sc.d	0110011	011	0001100
I-type	lb	0000011	000	n.a.
	lh	0000011	001	n.a.
	lw	0000011	010	n.a.
	lbu	0000011	100	n.a.
	lhu	0000011	101	n.a.
	addi	0010011	000	n.a.
	slli	0010011	001	0000000
	xori	0010011	100	n.a.
	srlr	0010011	101	0000000
	srai	0010011	101	0100000
	ori	0010011	110	n.a.
	andi	0010011	111	n.a.
	jalr	1100111	000	n.a.

Addressing Assumptions

1. We run our code on some “processor”
2. Our processor can execute “instructions” from a given set (i.e. addition, memory access, etc)
3. Our code can easily be transformed into a set of these “instructions”
4. The time it takes to execute each different “instruction” is roughly similar

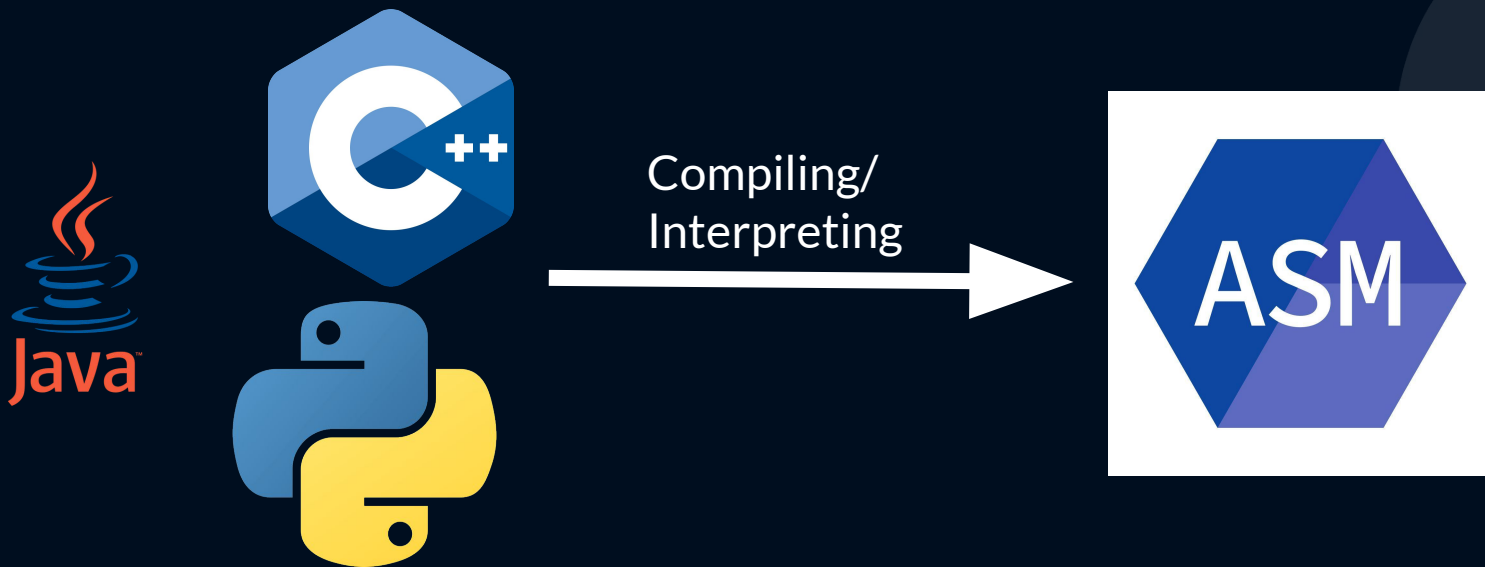
Addressing Assumptions

1. We run our code on some “processor”
2. Our processor can execute “instructions” from a given set (i.e. addition, memory access, etc)
3. Our code can easily be transformed into a set of these “instructions”
4. The time it takes to execute each different “instruction” is roughly similar

Compiling/Interpreting

Already covered in last lecture :D

Much more to it, but we know that with compiling/interpreting we can translate



Addressing Assumptions

1. We run our code on some “processor”
2. Our processor can execute “instructions” from a given set (i.e. addition, memory access, etc)
3. Our code can easily be transformed into a set of these “instructions”
4. The time it takes to execute each different “instruction” is roughly similar

Addressing Assumptions

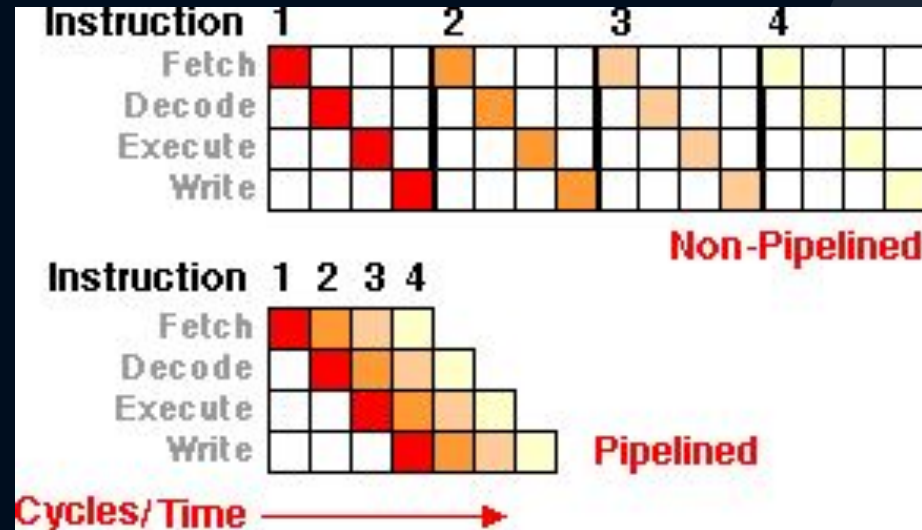
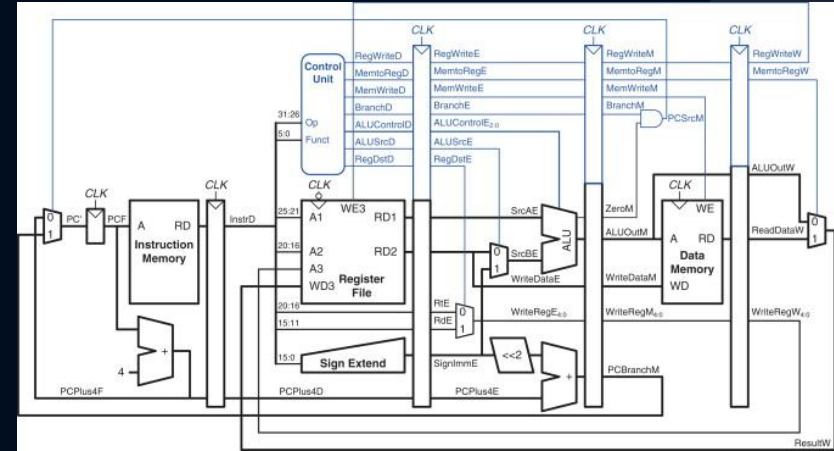
1. We run our code on some “processor”
2. Our processor can execute “instructions” from a given set (i.e. addition, memory access, etc)
3. Our code can easily be transformed into a set of these “instructions”
4. The time it takes to execute each different “instruction” is roughly similar

Addressing Assumptions

Processor = CPU

Pipelined into stages!

1. Instruction Fetch
2. Decode
3. Execute
4. Memory
5. Writeback



Addressing Assumptions

1. We run our code on some “processor”
2. Our processor can execute “instructions” from a given set (i.e. addition, memory access, etc)
3. Our code can easily be transformed into a set of these “instructions”
4. The time it takes to execute each different “instruction” is roughly similar

Addressing Assumptions

1. We run our code on some “processor”
2. Our processor can execute “instructions” from a given set (i.e. addition, memory access, etc)
3. Our code can easily be transformed into a set of these “instructions”
4. The time it takes to execute each different “instruction” is roughly similar

Lowkey I lied

Yeah, that's definitely not true...

Today's lecture will be mostly about how to take advantage of that “roughly”

Back to Speed

At its core, we want to solve a minimization problem:

$$\min_{\text{program } P} \text{Time}(P)$$

(for any program P that matches our spec)

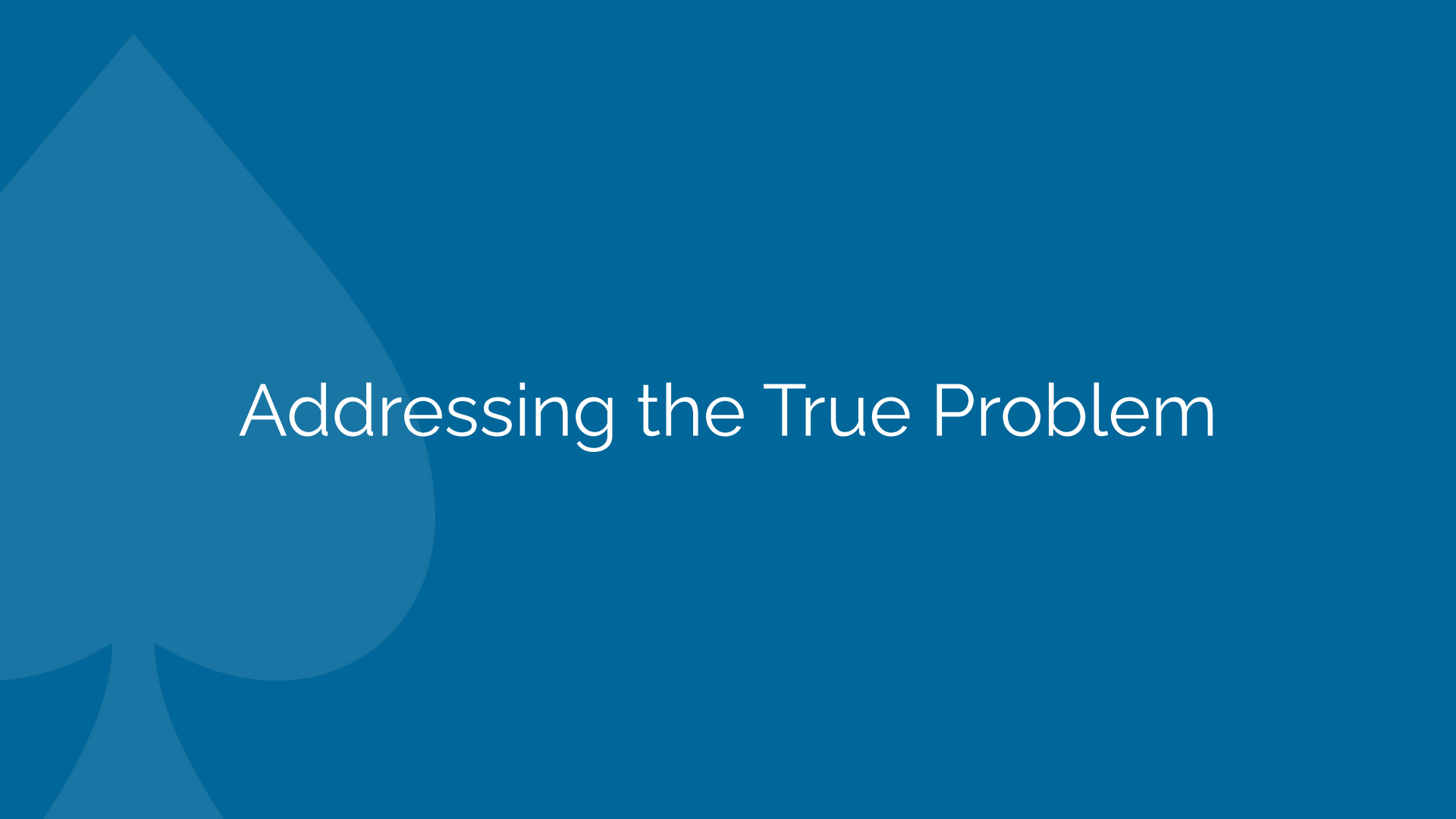
So the question is how does $\text{Time}(P)$ work?

- Processor has a set clock speed (i.e. 300 ns)
- Keeps running cycles, 'pushing' instructions forward
- $\text{Time}(P) = \text{num_cycles} * \text{clock_speed}$
- clock_speed is not something we can change (unless we change the hardware)
- **So we want to minimize num_cycles!**

Back to Speed

So how do we minimize the number of cycles?



The background is a solid dark blue. On the left side, there are several overlapping, semi-transparent shapes in a lighter shade of blue. These shapes include a large triangle pointing upwards and a large circle, creating a layered, abstract effect.

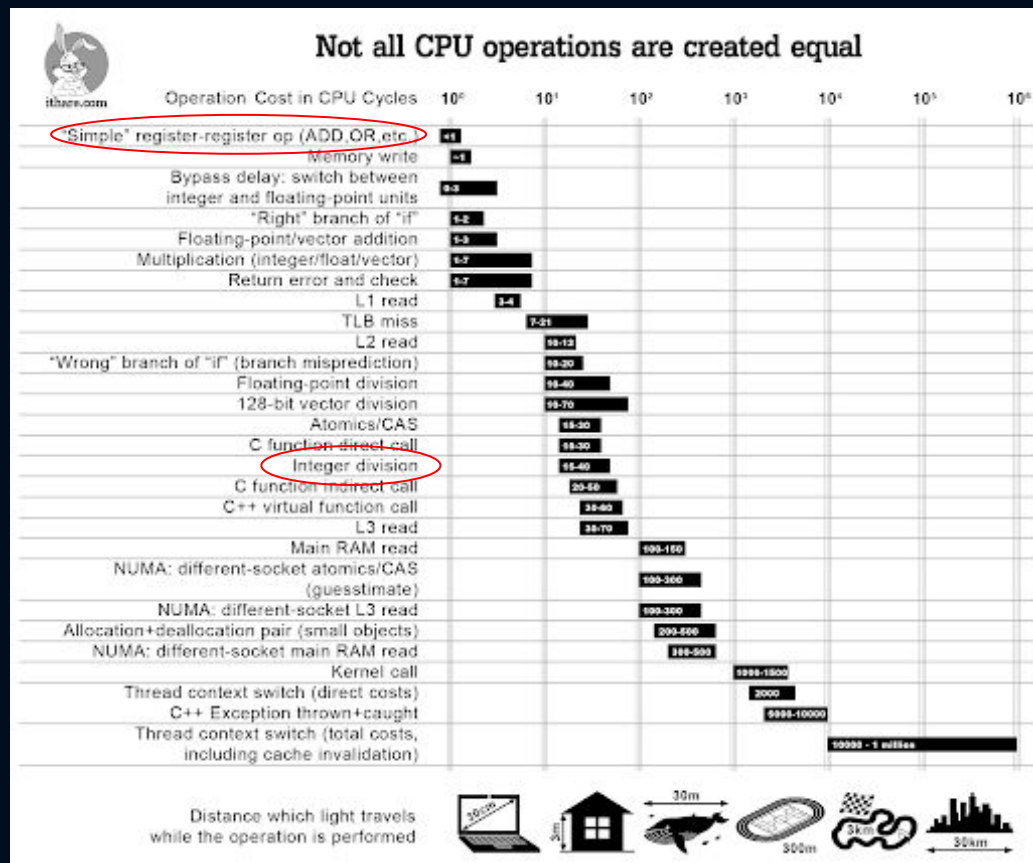
Addressing the True Problem

How Rough is “Roughly”

- Simple integer arithmetic operations (i.e. addition, bit shift, etc) take 1 cycle
- Integer multiplication takes ~5 cycles
 - Pipelined well, in practice often fast
 - Compilers do an amazing job of rearranging and executing instructions for us
- Integer division takes ~25 cycles
 - Not as well pipelined, so in practice quite slow

Can we even get slower than this

How Rough is "Roughly"



The Real Culprit

- Division might be something to look out for
 - But there's a **much bigger culprit**
- **MEMORY**
 - Having lots of memory is expensive
 - Having a little is cheap
 - We want a lot, but also cheap!?!?
 - A frontier of caches

Caches are **awesome**

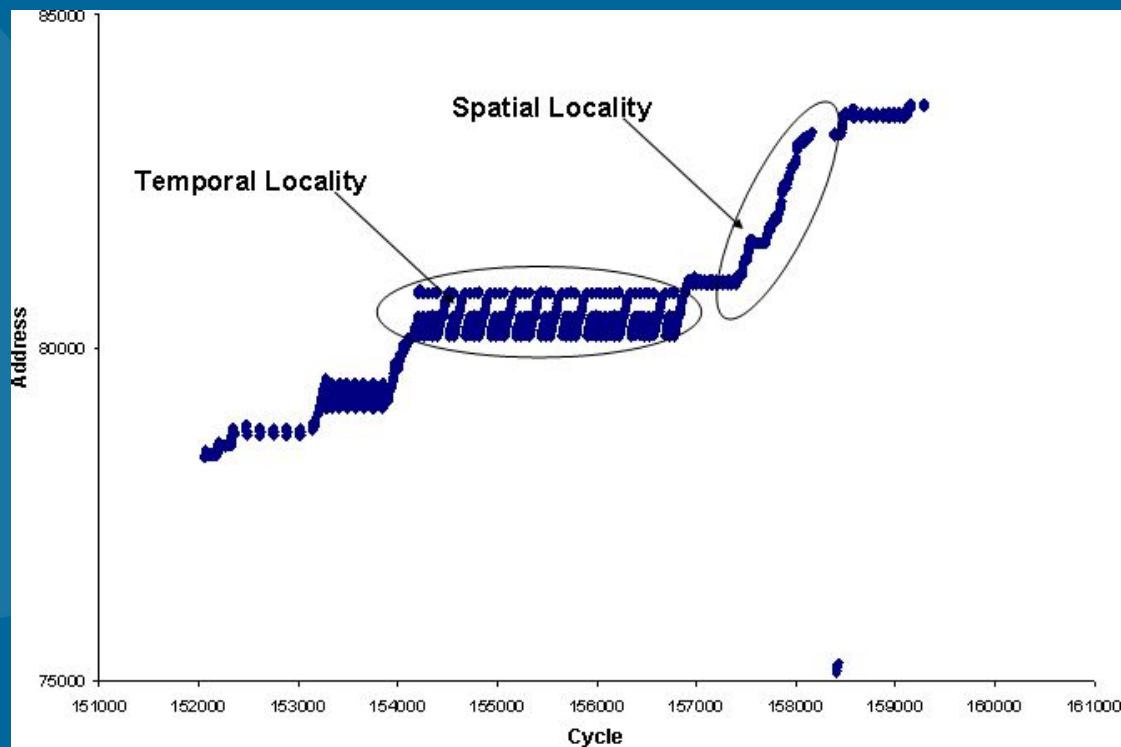
We should consider and utilize them!

A Typical Memory Hierarchy

- Everything is a cache for something else...

	Access time	Capacity	Managed By
On the datapath			
Registers	1 cycle	1 KB	Software/Compiler
↕			
Level 1 Cache	2-4 cycles	32 KB	Hardware
↕			
Level 2 Cache	10 cycles	256 KB	Hardware
↕			
On chip			
Level 3 Cache	40 cycles	10 MB	Hardware
↕			
Other chips			
Main Memory	200 cycles	10 GB	Software/OS
↕			
Flash Drive	10-100us	100 GB	Software/OS
↕			
Mechanical devices			
Hard Disk	10ms	1 TB	Software/OS

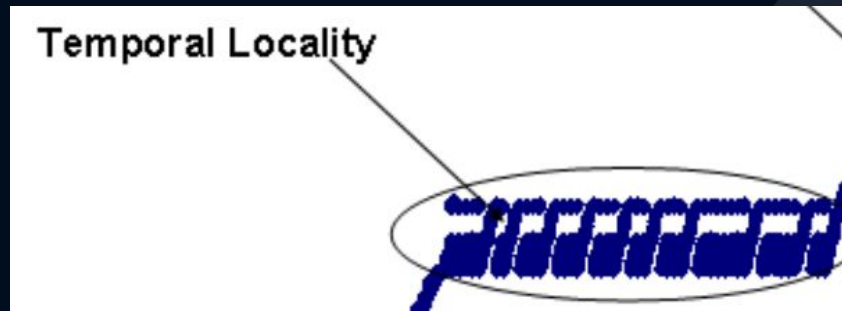
Locality



Temporal Locality

- Caches love reuse
 - Data already in cache -> cache hit (yay)
 - Otherwise -> cache miss (big no no)

Example:



Array of N numbers, Q queries from it where we just add $arr[i]$ to sum

- If query indices are uniform, reuse is low
- But suppose 95% of query indices are in a set containing 1% of the N elements
 - Tons of reuse!
 - In practice, can lead to $>2x$ speedup!

Spatial Locality

- Caches hate fetching
 - Want to fetch as rarely as possible
 - Fetch entire lines at a time
 - Close spatially



Pretend you are the cache

1. Program asks you for value at `arr[0]`
2. You don't have it, so you go fetch it from the next level up
3. I mean... while you're there, might as well fetch `arr[1]`, `arr[2]`, and `arr[3]` as well c:
4. You come back, and whaddayaknow
 - a. *The program asks for `arr[1]`*
5. You're prepared (cache hit) and immediately return it like a boss

Spatial Locality

Example:

You have an $N \times M$ matrix and want to sum over it

Note: memory layout is row-order, so $[[1, 2], [3, 4]]$ will be stored as $[1, 2, 3, 4]$ in memory

Approach 1: Outer loop over rows, inner over columns

Approach 2: Outer loop over columns, inner over rows

Which one should we go with?

Spatial Locality

Example:

You have an $N \times M$ matrix and want to sum over it

Note: memory layout is row-order, so $[[1, 2], [3, 4]]$ will be stored as $[1, 2, 3, 4]$ in memory

Approach 1: Outer loop over rows, inner over columns

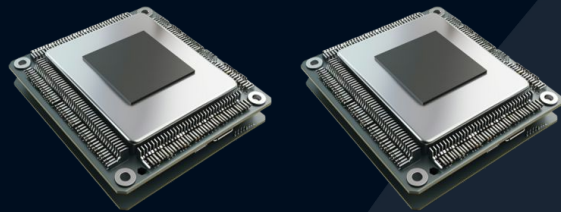
Approach 2: Outer loop over columns, inner over rows

Which one should we go with?

In practice, for a 5000 x 5000 matrix, I got a 4.5x difference!

Memory Level Parallelism

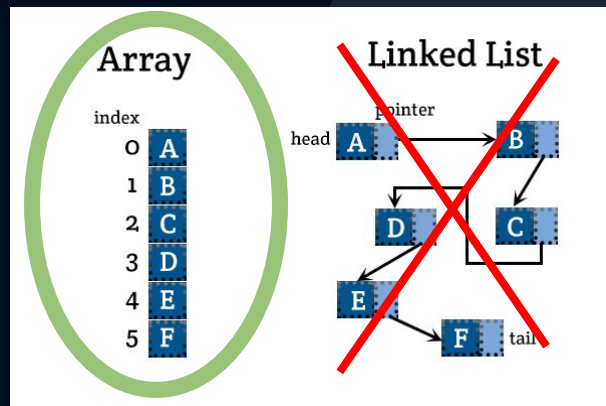
TL;DR: modern caches are super optimized



- Even though misses from a cache can take a while, modern caches are able to handle multiple requests at once

Example: summing over N values (here $N = 10^7$)

1. **Best:** Accessing array indices in order: `arr[i]`
 - a. ~11ms, also highly optimizable by compiler
2. **Meh:** Accessing array indices randomly: `arr[idx[i]]`
 - a. ~100ms, can still parallelize requests
3. **Really Bad:** Linked List with random addresses: `node->val`, `goto node->next`
 - a. ~1330ms, have to trace pointers in order





Dealing with a Bottleneck

PROFILE TIME!!!

But this time I mean it!

One nice way to profile:

1. Have readable and nice code (often overlooked)
2. Profile every major function; for each, measure:
 - a. Time spent solely inside this function
 - b. Time spent inside this function including subcalls
 - c. Number of times we called this function
 - d. (maybe more)

With this system, we will know which functions we should improve!

Compute-Bound vs. Memory-Bound

- Once we identify a piece of code to improve, we can classify it
 - Compute-bound
 - Memory-bound

Compute-bound = lots of waiting for CPU to finish math

Memory-bound = lots of waiting for CPU to fetch memory

(kind of like finding a bottleneck within our bottleneck)

It's Kinda Just Overcooked

- Compute-bound = mostly waiting for math to happen, memory loading is not an issue
- Similar to having tons of ingredients ready in Overcooked, but being bottlenecked by cooking



It's Kinda Just Overcooked

- Memory-bound = mostly waiting for memory fetching, math is not an issue
- Similar to waiting for ingredients to come your way in Overcooked, unable to cook anything in the meantime



Addressing Our Bottleneck

Now that we have a classified bottleneck, how can we address it?

- **Memory-bound:**
 - Reuse data **(read less bytes)**
 - Change data layout **(read less bytes)**
 - Reduce precision **(read less bytes)**
- **Compute-bound:**
 - Reduce amount of computation **(run less instructions)**
 - Algorithmic improvements
 - Approximations, pruning, sparsity
 - Reduce cost of computation **(run cheaper instructions)**
 - Vectorization
 - Reduce dependencies, unroll loops
 - Bit tricks
 - Lookup tables (careful to not run into memory-bound issues)



Takeaways

Practical Takeaways

- The cache is there to help, so use it
- Profile often and with care
- Keep your data flat where possible
 - Avoid chasing pointers (i.e. linked lists)
- Don't overuse the memory
- Precomputation can work wonders
- Fast code makes the CPU wait less

Lunch 🍷

Leave any type of feedback at pkr.bot/feedback!

tropical
SMOOTHIE
CAFE®



Practical Example

All of This Sounds Cool in Theory, I Guess

Let's walk through a poker specific example.

Recall Paco's lecture on CFR

```

double CFR::traverse(int player, const GameState& state, double pi_i, double pi_opp, int t) {
    if (state.is_terminal()) {
        return state.get_rewards(player);
    }

    if (state.is_chance_node()) {
        unique_ptr<GameState> next_state = state.sample_chance_node(rng);
        return traverse(player, *next_state, pi_i, pi_opp, t);
    }

    int current_player = state.get_active_player();

    if (current_player != player) {
        InfoSet& infoset = get_InfoSet(current_player, state);
        auto [sampled_action, sample_prob] = infoset.sample_regret_action(rng);
        unique_ptr<GameState> next_state = state.next_game_state(sampled_action);
        return traverse(player, *next_state, pi_i, pi_opp * sample_prob, t);
    }

    if (current_player != player){
        throw logic_error("This should not be possible");
    }
}

```

...

```

...
InfoSet& infoSet = get_InfoSet(player, state);

vector<pair<Action, double>> actions_with_probs = infoSet.get_action_w_probs();
double node_util = 0.0;

vector<double> action_deltas;
action_deltas.reserve(actions_with_probs.size());

for (const pair<Action, double>& act_and_prob : actions_with_probs) {
    unique_ptr<GameState> next_state = state.next_game_state(act_and_prob.first);
    double action_util = traverse(player, *next_state, pi_i * act_and_prob.second, pi_opp, t);
    node_util += act_and_prob.second * action_util;
    action_deltas.push_back(action_util);
}

for (size_t i = 0; i < action_deltas.size(); ++i) {
    action_deltas[i] = pi_opp * (action_deltas[i] - node_util);
}

infoSet.update_regret(action_deltas, t);
infoSet.update_average_strategy(pi_i, t);

return node_util;
}

```


Bottlenecks

- Lots of code, but we profiled and identified some bottlenecks

1. Allocating and populating lots of new GameStates!

Fix: an apply/undo system where we only save what we need

```
unique_ptr<GameState> next_state = state.sample_chance_node(rng);  
return traverse(player, *next_state, pi_i, pi_opp, t);
```



```
ChanceUndo undo;  
state.write_chance_undo(undo);  
state.apply_chance(rng);  
double util = traverse(player, state, pi_i, pi_opp, t);  
state.undo_chance(undo);  
return util;
```

Bottlenecks

- Lots of code, but we profiled and identified some bottlenecks

2. Allocating new vectors to store temporary values

Fix: we noticed traversal has recursive tree structure with low depth, so we made a pool of preallocated vectors to avoid reallocation

```
vector<double> action_deltas;
```



```
VectorPool::DeltaBuffer delta_buf;  
vector<double>& action_deltas = delta_buf.get();
```

Resulting Code

```
double CFR::traverse(int player, GameState& state, double pi_i, double pi_opp, int t) {
    if (state.is_terminal_node()) {
        return state.get_rewards(player);
    }

    if (state.is_chance_node()) {
        ChanceUndo undo;
        state.write_chance_undo(undo);
        state.apply_chance(rng);
        double util = traverse(player, state, pi_i, pi_opp, t);
        state.undo_chance(undo);
        return util;
    }

    int current_player = state.get_active_player();

    if (current_player != player) {
        InfoSet& infoset = get_InfoSet(current_player, state);

        VectorPool::ProbsBuffer probs_buf;
        auto& probs = probs_buf.get();
        auto [sampled_action, sample_prob] = infoset.sample_regret_action(rng, probs);

        ActionUndo undo;
        state.write_action_undo(sampled_action, undo);
        state.apply_action(sampled_action);
        double util = traverse(player, state, pi_i, pi_opp * sample_prob, t);
        state.undo_action(undo);

        return util;
    }
    ...
}
```

Resulting Code

```
...
    InfoSet& infoset = get_InfoSet(player, state);

    VectorPool::ActionBuffer action_buf;
    VectorPool::ProbsBuffer probs_buf;
    VectorPool::DeltaBuffer delta_buf;
    vector<pair<Action, double>>& actions_with_probs = action_buf.get();
    vector<double>& probs = probs_buf.get();
    vector<double>& action_deltas = delta_buf.get();

    infoset.get_action_w_probs(actions_with_probs, probs);
    action_deltas.reserve(actions_with_probs.size());
    double node_util = 0.0;

    for (size_t i = 0; i < actions_with_probs.size(); i++) {
        const pair<Action, double>& act_and_prob = actions_with_probs[i];

        ActionUndo undo;
        state.write_action_undo(act_and_prob.first, undo);
        state.apply_action(act_and_prob.first);
        double action_util = traverse(player, state, pi_i * act_and_prob.second, pi_opp, t);
        state.undo_action(undo);

        node_util += act_and_prob.second * action_util;
        action_deltas.push_back(action_util);
    }
...
```

Resulting Code

```
...  
    for (size_t i = 0; i < action_deltas.size(); ++i) {  
        action_deltas[i] = pi_opp * (action_deltas[i] - node_util);  
    }  
  
    infoset.update_regret(action_deltas, t);  
    infoset.update_average_strategy(pi_i, probs, t);  
    infoset.update_last_t(t);  
  
    return node_util;  
}
```



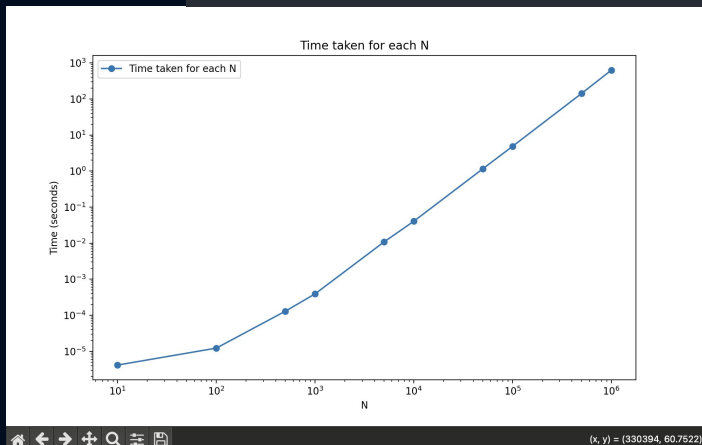
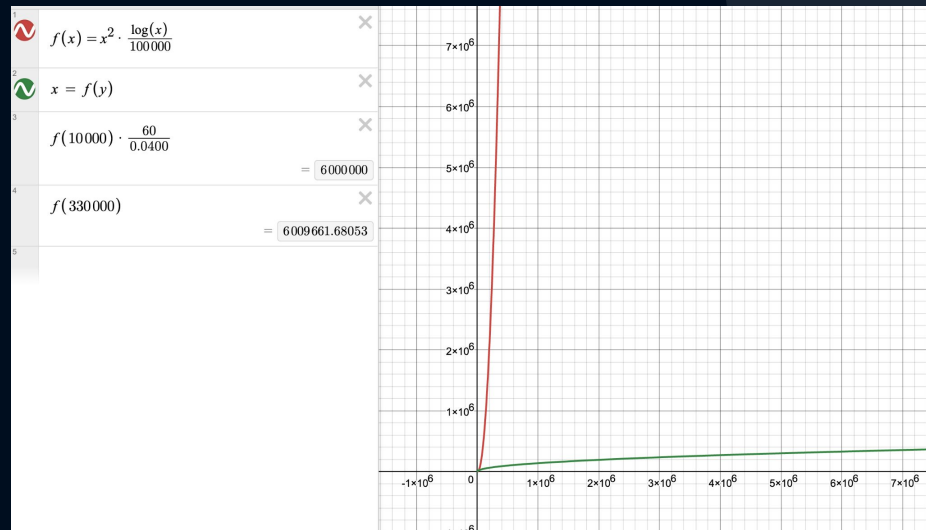
Giveaway Winner!

I Lost N Winner!

```
def f(n):  
    s = 1  
    for i in range(n):  
        if i % 2 == 0:  
            s *= i * 3 + 1  
        else:  
            s *= i * 5 + 2  
    return s  
N = ???  
x = f(N)
```

$\sim O(N^2 \log N)$

10000: 0.0400s $\rightarrow$?: ~ 60 s



Answer:
336963

Winner:
rksalib

Guess:
386911

Thanks for watching!

Information hub at pkr.bot/hub

Slides/notes will be posted on pkr.bot/resources

Make sure to check pkr.bot/piazza for updates

Lecture recordings at pkr.bot/panopto

Leave feedback at pkr.bot/feedback!